Video Downloader master docs

Short, simple guide to everything built in Video Downloader: a free desktop app that downloads videos (MP4 up to 4K), audio (MP3) and HD thumbnails from YouTube, Instagram, TikTok, Facebook, X and 1000+ sites.

#	Section	What it covers
1	Product and screens	Main screen, video card, playlists, downloads list, settings, link catching, tray, first start
2	Download engine	yt-dlp, FFmpeg, aria2, quality rules, MP3, thumbnails, turbo mode, progress, retries, file names
3	Front-end	Stack, folders, data flow, components, theme, tests
4	Desktop app and API	Electron main process, window and tray, IPC API, saved data, accounts, updates
5	Security, setup and release	Safety rules, variables, running, testing, building installers, releasing, limits

The project at a glance

- `src/` : the screens (Next.js 16 + React 19 + Tailwind CSS 4), exported as static files.
- `electron/` : the desktop app (Electron 45): window, tray, clipboard, accounts, updates.
- `electron/core/` : the download engine in plain Node.js (yt-dlp, FFmpeg, aria2), fully tested.
- `shared/` : types, the IPC API contract and formatting helpers used by both sides.
- `tests/` and `e2e/` : unit tests, real download tests and tests that drive the real app.
- `scripts/` : build helpers (tool download, Electron bundle, icons, these docs).
- `docs/` : these docs. Rebuild the PDF with `npm run docs`.

Current version

- **1.1.0**: adds captions (inside the video, always visible, or `.srt`) and English auto-translated captions.
- Windows installer: `Video-Downloader-Setup-1.1.0.exe`. Mac app: `Video-Downloader-1.1.0-arm64.dmg`.
- Download page: https://github.com/talal4a/video-downloader/releases/latest

Product and screens

What a person sees and can do in Video Downloader, screen by screen.

What Video Downloader is

- A desktop app for Windows (and macOS) that saves online videos to the computer.
- Paste or copy a link; the app shows the video and every quality; one click downloads it.
- Free and open source. No account, no ads, no paid services. Powered by yt-dlp, FFmpeg and aria2.

Supported sites

YouTube (videos, Shorts, playlists, channels), Instagram (posts, reels, stories), TikTok, Facebook, X (Twitter), Vimeo, Dailymotion, Reddit, Twitch, SoundCloud, Pinterest, LinkedIn, Threads, Bilibili, Rumble, Streamable, Bandcamp, VK, OK.ru and 1000+ more (everything yt-dlp supports), plus direct links to media files (`.mp4`, `.m3u8`, `.mpd`, `.mp3`…).

Main screen

Part	What it does
Header	Logo, engine status chip (**Ready**, **Setting up…**, **Updating engine…** or **Problem**), **Settings** button
Link box	"Paste a video link here". Pasting a link loads it straight away. **Paste** reads the clipboard; **Get video** loads what is typed; **×** clears
"Works with" chips	The best-known supported sites
First-time steps	Three cards (copy a link, paste it, pick quality) shown until the first download
Video card	Appears when a link is loaded (see below)
Downloads list	Every download with live progress and actions (see below)

- **Ctrl+V / Cmd+V** anywhere on the page (outside a text box) loads a copied link.
- **Drag and drop** a link (from a browser address bar) onto the window loads it.
- **Right-click** in the link box gives Cut / Copy / Paste / Select all. Right-clicking a link gives **Download this link**.
- While loading, a skeleton card shows "Getting video details from …".

Video card (single video)

- Big thumbnail with duration and its real size (for example `1280×720`), site badge, title, channel, length, views and **Open page**.
- **Save thumbnail (HD)** saves the biggest picture available as JPG (WebP is converted).

- **Video (MP4)** or **Audio (MP3)** switch. Video is greyed out for audio-only links.
- **Quality chips** list only what the video really has: 8K, 4K, 2K, 1080p (Full HD), 720p (HD), 480p, 360p, 240p, 144p, each with an estimated size. The top one has a **Best** badge. Vertical videos (Shorts, Reels) are labelled by their short side, so 1080×1920 is "1080p".
- The default choice is the best quality up to the setting **Default video quality** (1080p unless changed).
- **MP3 quality**: 320 kbps (Best), 192 kbps (Good), 128 kbps (Small), with estimated sizes.
- **Captions** (video only): pick a language, an auto-generated track, or **English (auto-translated)** for YouTube videos in other languages. Then choose how to add them:
  - **In the video** (default, fast): a captions track inside the MP4 that can be switched on/off in VLC, QuickTime, Windows Media Player…
  - **Always visible**: drawn onto the picture, so they show everywhere, also when the video is shared on WhatsApp. The video is re-encoded (H.264), which takes longer; progress shows "Adding captions to the video… 45%".
  - **Separate .srt file**: saved next to the video.
  - The last choice is remembered. Videos without captions show "No captions available for this video." The button then reads "Download MP4 · 1080p · CC".
- A watch link that is part of a playlist shows "This video is part of a playlist. **Show whole playlist →**".
- Live streams and premieres that have not started show a note instead of the download button.
- The big button reads "Download MP4 · 1080p" or "Download MP3 · 320 kbps" with "About 241 MB". After clicking it shows "Added to downloads" and the page scrolls to the list.

Playlist and multi-video posts

- **Playlists / channels** (each item is its own link): list of items with check boxes, **Select all / None**, kind (MP4/MP3) and quality (**Best** or a maximum such as 1080p, "or lower if missing"). Each item becomes its own download and is saved in a folder named after the playlist. A channel link opens its **Videos** tab.
- **Posts with several videos** (Instagram carousel, a page with several clips): downloaded in one go as `Title - 01.mp4`, `Title - 02.mp4`…
- Up to 400 items are shown; more are still included.

Downloads list

State	What it shows
Waiting in line	Queued (3 at a time by default)
Starting / retrying	"Starting…", "Retrying in normal mode…", "Updating the download engine…", "Connection problem — retrying…"
Downloading	Progress bar, percent, "Downloading video / audio", "12 MB of 80 MB", speed, time left, **Turbo** badge
Processing	Green moving bar: "Joining video and audio…", "Converting to MP3…", "Adding captions…", "Adding captions to the video… 45%", "Adding cover picture…"

State	What it shows
Saved	Green tick and file name; **Play**, **Show in folder**
Failed	Plain message and hint, **Show technical details**, **Retry**
Canceled	**Retry**

- Header: number of active downloads, **Open folder**, **Clear finished**.
- **×** cancels a running download (all helper programs are stopped, temporary files removed). The bin icon removes an item from the list; the file stays on disk.
- The format line shows captions too, for example "MP4 · 1080p · Captions (in video)".
- Warnings in amber, for example "Captions are not available for this video, so it was saved without them." or "The cover picture could not be added to this MP3."
- The list is kept between restarts (last 500). Downloads running when the app was closed show "The download stopped because the app was closed" with **Retry**.
- Windows taskbar icon shows the overall progress.

IDM-style link catching

- While the app runs it checks the clipboard every second.
- When a video link is copied anywhere (browser, WhatsApp, YouTube app…):
  - nothing on screen yet: the app loads it at once;
  - something already on screen: a banner "You copied a video link: youtube.com" with **Get this video** / **Dismiss**;
  - window hidden or not focused: a notification "Video link copied — Click here to download it."
- Only real video links count (a YouTube home page or an Instagram profile does not).
- Can be turned off in Settings or from the tray menu.

Tray, notifications and closing

- Closing the window keeps the app running near the clock (system tray) so it can catch links and finish downloads. The first time, a notification explains this.
- Tray menu: **Open Video Downloader**, **Download link from clipboard**, **Open downloads folder**, **Catch copied video links** (on/off), **Quit**.
- With "Keep running in the background" off, closing during downloads asks "Downloads are still running. Keep downloading / Quit anyway".
- Notifications when a download finishes or fails (when the window is not focused). Clicking "Download complete" shows the file.
- Only one copy of the app runs; opening it again brings the window to the front.

Settings panel

Group	Options
Downloads	Save folder (**Change…**, **Open**), default video quality (Best, 4K, 2K, 1080p, 720p, 480p, 360p), start with Video or Audio, downloads at the same time (1-5), **Turbo mode** with 4 / 8 / 16 connections
Convenience	Catch copied video links, keep running in the background, start with the computer, notifications, appearance (Auto / Light / Dark)
Accounts (optional)	Log in / Log out for Instagram, Facebook, TikTok, X (see Desktop app and API)
Download engine	yt-dlp version, FFmpeg, aria2 turbo, YouTube helper (built-in / Deno / Node.js) status, last update check, **Update automatically every day**, **Update now**
About	App version, app update status (**Restart to update** when ready), project link

Escape or a click outside closes the panel.

First start

- If a helper program is missing (an installer built without them, or an antivirus removed one), a banner "Getting ready for first use — this happens only once" downloads it with a progress bar: yt-dlp, FFmpeg, aria2 (Windows) and, only if needed, Deno.
- If that fails: a red banner names what is missing with a **Fix it** button.

Error messages

Situation	Message
YouTube bot check	"YouTube is asking to confirm you are not a robot." Wait, or restart the router
Login needed	"This site needs you to be logged in to get this video." → **Open Accounts**
Private / removed / blocked	"This video is private." / "…not available…" / "…blocked in your country."
Age-restricted	"This video is age-restricted and needs a signed-in account."
DRM (Netflix, Spotify…)	"This video is protected (DRM) and cannot be downloaded."
Not a video page	"This link isn't supported." / "No video was found at this link."
Network / slow	"Can't connect to the site." / "The connection is too slow or timed out."
Site changed	"The site changed something…"; the engine updates itself and retries
Disk / folder	"Your disk is full." / "The app can't write to the download folder."

Every error also has **Technical details** with the original yt-dlp message.

Appearance

- Light and dark themes (Auto follows the computer). Purple brand colour, rounded cards, system font (Segoe UI on Windows).
- Window 1180×840, smallest 860×620.

Download engine

How a link becomes a file. Code: `electron/core/` (plain Node.js, no Electron, so it is fully tested).

Helper programs

Program	Job	License
yt-dlp	Reads the video page, lists formats, downloads	Unlicense (public domain)
FFmpeg + FFprobe	Joins HD video with audio, MP3 conversion, cover art, thumbnail conversion	GPL
aria2	Turbo mode: up to 16 connections per file	GPL
Electron's own Node.js	Solves YouTube's JavaScript challenges for yt-dlp (no extra download)	MIT
Deno (only if needed)	Backup JavaScript runtime for YouTube	MIT

`engine.ts` finds them in this order: test overrides (`VD_YTDLP` , `VD_FFMPEG_DIR` , `VD_ARIA2C`), the installer's `resources/bin` , the app's own `bin` folder, then the system `PATH` (plus `/opt/homebrew/bin` and `/usr/local/bin` on Mac).

- yt-dlp is copied from the installer into the app's data folder so it can update itself even when the app is installed in a protected folder. A newer bundled copy replaces an older one.
- **Self-repair:** if yt-dlp, FFmpeg or aria2 (Windows) is missing, `toolsetup.ts` downloads it from the official GitHub release on start. yt-dlp is checked against its `SHA2-256SUMS` file; Deno against its `.sha256sum` file. Zips are opened with a small built-in extractor (`unzip.ts`) that refuses `../` paths.
- **YouTube runtime self-test:** at start the app runs itself as Node.js (`ELECTRON_RUN_AS_NODE=1 … --permission -`) with a tiny script, exactly as yt-dlp will. If that fails it uses Deno, then a system Node.js 22+, and downloads Deno only as a last resort.

Getting video details

`info.ts` runs:

```
yt-dlp -J --flat-playlist --no-playlist --no-warnings [--cookies file] -- <link>
```

- Links are cleaned first: `https://` is added, `youtu.be/…` works, a channel page goes to its **Videos** tab.
- The JSON is turned into a `MediaInfo` : title, channel, site, length, views, thumbnails (best first), qualities with sizes, subtitles, live flag, playlist entries.
- Cached for 10 minutes. Time limit 120 s.

- On "site changed" errors (`extractor` , `http_403`) the engine updates yt-dlp once (at most every 10 minutes) and tries again. Network errors are retried once after 1.5 s.

Quality rules

Choice	yt-dlp command
Video, chosen quality	`-f "bv*+ba/b" -S "res:<value>,vcodec:h264,acodec:aac" --merge-output-format mp4 --remux-video mp4`
Video, Best	`-S "res,vcodec:h264,acodec:aac"` (highest resolution first)
MP3	`-f "ba/b" --extract-audio --audio-format mp3 --audio-quality 320K|192K|128K --embed-thumbnail --convert-thumbnails jpg`
Captions	`--write-subs` or `--write-auto-subs --sub-langs <lang> --convert-subs srt` , plus `--embed-subs` for "In the video"

- Up to 1080p the app picks **H.264 + AAC in MP4**, which plays everywhere.
- YouTube only offers 2K and 4K as VP9 or AV1; the app then picks **VP9** (better support than AV1). VP9 MP4 plays in VLC, IINA, Windows Media Player and browsers, but **not in QuickTime** on Mac.
- "Value" is the short side of the picture (yt-dlp's `res`), so vertical and cinema-shaped videos get the right label. Size estimates add the best AAC audio to the video size.

Downloading

Every download gets its own temporary folder `<temp>/VideoDownloader/<job id>-<attempt>` with plain ASCII names (`media.mp4` , or `001.mp4` , `002.mp4` for multi-video posts). Common options:

```
--ignore-config --color never --encoding utf-8 --cache-dir <app>/cache
--ffmpeg-location <ffmpeg folder> --no-js-runtimes --js-runtimes node:<app exe>
--socket-timeout 30 --retries 10 --fragment-retries 10 --extractor-retries 3
--newline --progress --progress-template … --ignore-errors --no-mtime
--embed-metadata --concurrent-fragments 8 --no-playlist|--yes-playlist
```

Turbo mode (on by default, needs aria2):

```
--downloader aria2c --downloader m3u8,dash:native
--downloader-args "aria2c:-x 16 -s 16 -k 1M --auto-save-interval=1"
```

Normal files are split over up to 16 connections; streamed formats (DASH/HLS) use 8 parallel fragments in yt-dlp itself.

Live progress

- yt-dlp prints one JSON line per update (`VDPROG {...}`): bytes done, total, speed, time left, format, item number.
- Post-processing prints `VDPP "<step>" "started"` , shown in plain words (Merger → "Joining video and audio…").
- aria2 keeps its own output until it ends, so in turbo mode the app reads aria2's control file (`*.aria2` , rewritten every second) every 0.7 s (`aria2.ts`) for bytes done, total, smoothed speed and time left.
- The bar is shared: video stream 0-92 %, audio stream 92-100 %; for several videos, item N of M. It never moves backwards.

Retries and safe mode

Problem	What happens (max 3 tries)
Turbo download failed	Try again without aria2 ("Retrying in normal mode…")
Site changed / 403 / format missing	Update yt-dlp once, then try again
Network, timeout, rate limit, unknown	Wait 4 s × try number, then try again
Cover art, subtitles or metadata failed	"Safe mode": try again without these extras, keep the video, show a warning
Private, removed, DRM, unsupported…	Stop at once with the plain message

If at least one finished file exists, the download counts as done.

Captions

Mode	How
In the video (`embed`)	yt-dlp puts the captions in the MP4 as a `mov_text` track. FFprobe checks the track is there; then the extra `.srt` is deleted. If it is missing, the `.srt` is kept with a warning
Always visible (`burn`)	After the download, FFmpeg draws the `.srt` onto the picture: `-vf subtitles=captions-1.srt:fontsdir=fonts:force_style='FontName=Arial' -c:v libx264 -preset veryfast -crf 20 -c:a copy` . It runs inside the job folder with simple names (no Windows path escaping problems) and gets a copy of the system font (Arial on Windows/Mac, DejaVu Sans on Linux) so text always shows. Progress comes from `-progress pipe:1` . Cancel stops it. On failure the original video and `.srt` are kept with a warning
Separate file (`file`)	The `.srt` is saved as `Title.<lang>.srt`

- Languages: the video's own captions, auto-generated captions in the original language (`<lang>-orig`), and **English (auto-translated)** when YouTube offers it and there is no English track.

- If captions fail (for example YouTube refuses them), "safe mode" saves the video without them and says so.

Saving files

- Finished files are moved to the download folder (default **Downloads → Video Downloader**), in a sub-folder for playlists.
- Names come from the title, made safe for Windows: `< > : " / \ | ? *` and control characters are replaced, names like `CON` are prefixed, length is capped at 120 characters. Urdu, Arabic and emoji are kept.
- Never overwrites: `Title.mp4`, `Title (1).mp4`, `Title (2).mp4` …
- Subtitles are saved as `Title.<lang>.srt`. Thumbnails as `Title - thumbnail.jpg`.
- Moving across drives falls back to copy + delete. The temporary folder is removed afterwards and cleaned on every start.

Thumbnails

- Candidates, best first: for YouTube `maxresdefault.jpg` and `hq720.jpg`, then yt-dlp's list sorted by preference and size, then smaller YouTube sizes. Relative links are made absolute.
- The first real image wins (tiny placeholders under 1.2 KB and 404s are skipped). The picture type is read from the bytes.
- JPG and PNG are kept; WebP, AVIF, GIF and HEIC are converted to high-quality JPG with FFmpeg.

Keeping yt-dlp fresh

- 8 s after start and every 3 hours, if more than 20 hours passed since the last check and no download is running: `yt-dlp --update-to stable`.
- Also on demand (**Update now**) and automatically after "site changed" errors.
- Result is stored in `engine.json` and shown in Settings.

Stopping a download

- **Cancel** stops yt-dlp and everything it started: `taskkill /T /F` on Windows, the whole process group on Mac/Linux.
- Helper windows are always hidden (no black console windows on Windows).

Front-end

The screens in `src/`. They are a Next.js app exported as static files and shown inside the desktop window.

Stack

Part	Library
UI	React 19
Framework	Next.js 16 (App Router), `output: 'export'` (static files in `out/`)
Styling	Tailwind CSS 4 (`@tailwindcss/postcss`) with a custom purple `brand` palette
Icons	`lucide-react`
Language	TypeScript 5.9
Lint	ESLint 9 with `eslint-config-next` (core-web-vitals + TypeScript)

No web server, no database, no outside fonts: everything works offline. Talking to the computer goes through the desktop API (`window.vd`).

Folder map

Path	Purpose
`src/app/layout.tsx`	Page shell; applies the saved theme before first paint (no white flash)
`src/app/page.tsx`	The only page; renders `App`
`src/app/globals.css`	Tailwind setup, `dark:` variant, brand colours, progress stripes, fade-in
`src/components/App.tsx`	Main screen: link box, banners, loading/error/video states, paste, drag and drop, events
`src/components/MediaCard.tsx`	Single video: thumbnail, Video/Audio switch, quality and MP3 chips, subtitles, download
`src/components/PlaylistCard.tsx`	Playlists and multi-video posts with selection
`src/components/DownloadList.tsx`	Downloads list rows, progress, actions, error details
`src/components/SettingsPanel.tsx`	Settings side panel
`src/components/Toasts.tsx`	Small pop-up messages (success, error, info) with an optional action
`src/components/ui.tsx`	Button, IconButton, Toggle, Segmented, Chip, ProgressBar, Card
`src/lib/api.ts`	`getApi()` for `window.vd`, link finder, class helper

Path	Purpose
`shared/types.ts`	Shared types: `MediaInfo`, `DownloadRequest`, `Job`, `Settings`, `EngineInfo`…
`shared/api.ts`	The `VideoDownloaderApi` contract and IPC channel names
`shared/format.ts`	Bytes, speed, duration, time left, view counts, quality labels

Data flow

- `getApi()` returns `window.vd`, which the preload script exposes. Opened in a normal browser, the page shows a short "open it with the app" screen instead.
- **Loading a link:** `App.load()` → `vd.getInfo(url)` → `{ ok, value | error }`. A request counter ignores late answers when a newer link was loaded.
- **Thumbnails** come back as a `data:` URL from `vd.getThumbnail(key)` (so sites that block hot-linking still show).
- **Downloads:** `vd.startDownloads([...])` adds jobs; live changes arrive through `vd.onDownloadUpdate(job)` and `vd.onDownloadRemoved(id)`; the list is merged by job id.
- **Live events also used:** `onClipboardLink`, `onOpenUrl` (notification, tray, right-click, second launch), `onSettingsChanged`, `onEngineInfo` (status and first-start progress), `onAppUpdate`.
- **Settings** are read with `vd.getSettings()` and changed with `vd.updateSettings(patch)`; the main process checks every value.
- **Theme:** the chosen mode is also kept in `localStorage` (`vd-theme`) only so the first paint uses the right colours; the real value lives in the app's settings file.

Screen states

State	Shows
`idle`	Link box and three "how it works" cards (until something is downloaded)
`loading`	Skeleton card "Getting video details from …"
`error`	Message, hint, **Try again**, **Open Accounts** for login errors, technical details
`ready`	`MediaCard` or `PlaylistCard`

Accessibility and comfort

- Real buttons, `role="radio"` chips, `role="switch"` toggles, labelled icon buttons, visible focus rings.
- Keyboard: Enter loads, Escape closes Settings, Ctrl/Cmd+V loads a copied link anywhere.
- Light and dark themes with matching contrast.

Testing and build

Command	What it does
`npm run build:ui`	`next build` → static files in `out/`
`npm run lint`	ESLint
`npm run typecheck`	TypeScript for the UI and the desktop app
`npm run test:e2e`	Drives the real app with Playwright (see Security, setup and release)

Elements used by tests: `url-input`, `go-button`, `paste-button`, `media-card`, `download-button`, `job` (with `data-status`), `info-error`, `open-settings`, `setup-banner`.

Desktop app and API

The Electron main process in `electron/`: it owns the window, the tray, the clipboard, saved data and the download engine, and offers a small API to the screens.

Stack

- Electron 45 (Chromium + Node.js 24), bundled with esbuild into `dist-electron/main.cjs` and `preload.cjs`.
- `electron-updater` for app updates from GitHub Releases. `electron-builder` for installers.
- Everything else is plain Node.js in `electron/core/`.

Files

File	Purpose
`main.ts`	Start-up, window, tray, menus, notifications, IPC handlers, settings, history, logging
`preload.ts`	Exposes `window.vd` safely (`contextBridge`)
`protocol.ts`	Serves the exported UI as `app://local/…` with a strict Content-Security-Policy
`clipboard.ts`	Link catching (checks every second)
`accounts.ts`	Optional site log-in and cookie export
`updater.ts`	App self-update (Windows installer builds)
`core/engine.ts`	Finds and updates tools, YouTube runtime self-test, common yt-dlp options
`core/toolsetup.ts`, `core/unzip.ts`	Downloads missing tools; ZIP extractor
`core/info.ts`	Video details and thumbnails
`core/downloads.ts`	Queue, jobs, progress, retries, saving files
`core/ytdlp.ts`	yt-dlp command lines, JSON parsing, progress parsing
`core/aria2.ts`	Reads aria2's control file for live turbo progress
`core/errors.ts`	Turns yt-dlp errors into plain messages
`core/files.ts`, `core/urls.ts`, `core/thumbnails.ts`, `core/cookies.ts`, `core/settings.ts`, `core/store.ts`, `core/process.ts`	Safe names, link detection, images, cookie files, settings rules, JSON files, running and stopping programs

Window and start-up

- One copy only (`requestSingleInstanceLock`); a second launch focuses the window and passes on any link.
- Window opens when ready (no white flash) with the system's light/dark background.
- In development it loads `http://localhost:3123`; in the app it loads `app://local/`.
- Links to other sites open in the normal browser; the window cannot be navigated away.
- **Start with the computer** starts hidden (`--hidden`) in the tray.
- Windows: no menu bar. Mac: standard App / Edit / View / Window menus.
- Start-up order: clean the temporary folder → find tools → download missing tools → YouTube runtime self-test → engine status to the UI. The window does not wait for this; requests wait for the engine to be ready.

IPC API (`window.vd`)

Each call returns a promise. Errors come back as `{ ok: false, error: { code, message, hint, details } }`.

Call	Purpose
`getInfo(url)`	Video details (`MediaInfo`)
`getThumbnail(key)` / `saveThumbnail(key)`	Preview as `data:` URL / save HD JPG to the download folder
`startDownloads(requests)`	Add one or more downloads (max 2000); every request is checked
`listDownloads()`, `cancelDownload(id)`, `retryDownload(id)`, `removeDownload(id)`, `clearFinished()`	Manage the list
`openFile(path)`, `showInFolder(path)`, `openDownloadsFolder()`, `openExternal(url)`	Open things (only known download files; only `http(s)` links)
`readClipboardUrl()`	First link in the clipboard
`getSettings()`, `updateSettings(patch)`, `chooseDownloadDir()`	Settings
`getEngineInfo()`, `updateEngine()`	Tool status; update yt-dlp or repair missing tools
`getAccounts()`, `login(id)`, `logout(id)`	Optional site accounts
`getAppInfo()`, `getAppUpdate()`, `checkAppUpdate()`, `installAppUpdate()`	Version and self-update

Events: `onDownloadUpdate`, `onDownloadRemoved`, `onClipboardLink`, `onOpenUrl`, `onSettingsChanged`, `onEngineInfo`, `onAppUpdate`. Download updates are sent at most 4 times a second per job.

Data model

DownloadRequest: `url`, `title`, `thumbnail`, `site`, `duration`, `kind` (`video` / `audio`), `quality` (number or `best`), `audioBitrate` (320/192/128), `subtitles` (`{ lang, auto, mode }` with mode `embed`, `burn` or `file`, or none), `subfolder`, `multi`.

Job: `id`, `request`, `status` (`queued`, `starting`, `downloading`, `processing`, `done`, `error`, `canceled`), `stage`, `progress` (0-1), `downloadedBytes`, `totalBytes`, `speed`, `eta`, `itemIndex` / `itemCount`, `filePath`, `files`, `error`, `warnings`, `turbo`, `attempts`, times.

Settings (defaults): download folder `Downloads/Video Downloader`, catch links **on**, turbo **on** with 16 connections, 3 at a time, keep running in tray **on**, start with computer **off**, start with **Video**, default quality **1080p**, MP3 **320 kbps**, theme **Auto**, notifications **on**, daily engine update **on**, captions **In the video**. Bad values are corrected (for example connections 1-16, at a time 1-5).

Saved data

App data folder: `%APPDATA%\Video Downloader` on Windows, `~/Library/Application Support/Video Downloader` on Mac.

File / folder	Contents
`settings.json`	Settings
`history.json`	Last 500 downloads (written safely: temp file then rename)
`engine.json`	Last yt-dlp update time and result
`bin/`	Updatable yt-dlp, downloaded FFmpeg (`bin/ffmpeg/`), aria2, Deno
`cache/`	yt-dlp cache (YouTube player code)
`cookies/`	Exported site cookies (only for logged-in sites)
`logs/main.log`	App log, rotated at 2 MB

Accounts (optional log-in)

Site	Log-in page	Logged in when cookie exists
Instagram	instagram.com/accounts/login	`sessionid`
Facebook	facebook.com/login	`c_user`
TikTok	tiktok.com/login	`sessionid`
X (Twitter)	x.com/i/flow/login	`auth_token`

- The person logs in inside a normal browser window with its own storage (`persist:acct-<site>`); the window closes by itself once logged in. The app never sees or stores the password.

- For a link from that site only, cookies are written to `cookies/<site>.txt` (Netscape format, owner-only permissions) and given to yt-dlp with `--cookies`.
- **Log out** clears that site's storage and deletes the file.
- YouTube log-in is not offered: Google blocks sign-in inside apps.

App updates

- Only in installed Windows builds. Checks 15 s after start and every 6 hours from https://github.com/talal4a/video-downloader/releases (needs the repository to be public and the release to include `latest.yml`).
- Downloads in the background; a banner "A new version is ready" offers **Restart now**. Otherwise it installs when the app quits.
- Errors are silent (no scary messages for the user).
- The person can also press **Check now** in Settings → About.
- How an installed app finds a new version: it reads `latest.yml` from the newest GitHub release, compares the version with its own, downloads the new installer (checked against the SHA-512 in `latest.yml`), then installs it on restart. Settings, download history and files are kept.

Logging

- `logs/main.log`: start-up, tools found, every download attempt, failures with yt-dlp details, update results, crashes. Useful when someone sends a problem report.

Security, setup and release

How the app keeps people safe, and how to run, test, build and release it.

Security measures in the code

- **Locked-down window:** `contextIsolation`, `sandbox` and no Node.js in the page. The page only gets the small `window.vd` API.
- **Content-Security-Policy** on the app's own pages (`protocol.ts`): scripts and styles only from the app; images may come from `https:`; no plugins, no framing.
- **No navigation away:** links to other sites open in the normal browser; pop-ups are blocked.
- **Checked input:** every download request is re-checked in the main process (link must be `http(s)`, kind, quality 100-4320, bitrate, subtitle code, sizes). Settings are corrected to safe values.
- **Files: Play / Show in folder** only work for files in the download folder or made by a download. Only `http(s)` links can be opened externally.
- **Downloads from the internet:** yt-dlp and Deno are checked against their official SHA-256 sums; zips with `../` paths are refused; all tools come from their official GitHub releases.
- **No shell:** programs are started with an argument list (never through a command shell), so titles and links cannot run commands. `--` comes before every link.
- **Accounts:** passwords are typed into the site itself; cookies are stored per site with owner-only permissions and used only for that site.
- **Privacy:** no analytics, no tracking, no server. The app only talks to the video sites and to GitHub (updates).
- **Safe file names** for Windows, and never overwriting existing files.

Environment variables (developers and tests only)

Normal users never need these.

Name	Meaning
`VD_USER_DATA`	Use another app data folder
`VD_DOWNLOAD_DIR`	Force the download folder
`VD_DISABLE_CLIPBOARD`	Turn off link catching
`VD_DISABLE_UPDATER`	Turn off app updates
`VD_DEV_URL`	Dev server address (default `http://localhost:3123`)
`VD_YTDLP`, `VD_FFMPEG_DIR`, `VD_ARIA2C`	Use specific tools (`VD_ARIA2C=` empty turns turbo off)
`VD_TOOLS_BASE`	Download missing tools from a mirror (tests)
`VD_FORCE_JS_FALLBACK`	Pretend the built-in YouTube runtime fails (tests)

Name	Meaning
`VD_E2E_APP`, `VD_TEST_YOUTUBE`, `VD_TEST_VERBOSE`	Test the installed app, run internet YouTube tests, verbose logs

Run locally (Mac)

1. Install Node.js 22.12 or newer (`brew install node`) and, for full features, `brew install ffmpeg aria2`.
2. `git clone https://github.com/talal4a/video-downloader.git` and `cd video-downloader`.
3. `npm install`
4. `npm run setup` downloads yt-dlp for this computer into `resources/bin/mac`.
5. `npm run dev` opens the app with live reload (Next.js on port 3123 + Electron).

Commands

Command	What it does
`npm run setup`	Download yt-dlp (+ FFmpeg and aria2 on Windows) into `resources/bin/<os>`
`npm run dev`	Development app with live reload
`npm run build`	UI to `out/` + desktop code to `dist-electron/`
`npm start`	Run the built app
`npm run lint` / `npm run typecheck`	Code checks
`npm test`	Unit tests
`npm run test:integration`	Real downloads against a local test server
`npm run test:e2e`	Drive the real app (add `VD_E2E_APP=<path to exe>` to test an installed build)
`npm run dist:win`	Windows installer `release/Video-Downloader-Setup-<version>.exe` (works on a Mac, no Wine needed)
`npm run dist:mac`	Mac app `release/Video-Downloader-<version>-arm64.dmg` (ad-hoc signed)
`npm run dist:linux`	Linux AppImage
`npm run icons`	Redraw the app and tray icons
`npm run docs`	Rebuild this PDF (`docs/Video-Downloader-Docs.pdf`)

Tests

Suite	Count	What
Unit (`tests/unit`)	75	Link detection, error messages, safe names, quality labels and sizes, yt-dlp commands, progress parsing, aria2 control file, cookies, settings, ZIP extractor
Integration (`tests/integration`)	24 (+1 internet)	Real yt-dlp, FFmpeg and aria2 against a local server: 720p/best merge, MP3, thumbnails, captions (inside the video, `.srt`, drawn onto the picture and checked by pixel brightness), Unicode names, `(1)` copies, turbo with live progress, cancel, multi-video posts, parallel queue, errors, first-start tool download, checksum refusal, YouTube runtime and Deno fallback
End-to-end (`e2e`)	11	Starts the real app: paste → download 720p, MP3 + thumbnail, captions inside the video, clipboard catching, live progress + cancel, error screen, settings, dark mode, Node.js self-test of the installed app, real YouTube (internet)

- The local server (`tests/helpers/media-server.ts`) supports Range requests and a slow mode; test videos are made with FFmpeg (`tests/helpers/fixtures.ts`).
- Checked on Node.js 22 and 26. The E2E suite was also run against the packaged app.

Building the installers

- `scripts/fetch-binaries.mjs` downloads the official tools: yt-dlp (checksum-verified), FFmpeg (yt-dlp or BtbN builds, shared DLL version first) and aria2 1.37 for Windows; on Mac it uses Homebrew FFmpeg/aria2 when present. It unzips with the system tool (`ditto` , `tar.exe` , `unzip`).
- `electron-builder.yml` :
  - Windows: one-click NSIS installer, per-user (no admin password), desktop and Start-menu shortcuts, opens after install, executable `VideoDownloader.exe` .
  - Mac: `.dmg` , ad-hoc signature (free), no hardened runtime.
  - Linux: AppImage.
  - Tools go to `resources/bin` , icons to `resources/icons` ; the UI and code are packed in `app.asar` .
- The installer is not code-signed (that costs money), so Windows shows "Windows protected your PC" once: **More info** → **Run anyway**.

Releasing a new version (manual)

GitHub Actions is blocked on this account by a billing lock, so releases are made by hand:

1. Change `"version"` in `package.json` (for example `1.2.0`).
2. On the Mac: `npm run dist:win` (and `npm run dist:mac` if wanted).
3. Open `https://github.com/talal4a/video-downloader/releases/new?tag=v1.2.0&title=Video%20Downloader%201.2.0` .

4. Attach from `release/`: `Video-Downloader-Setup-1.2.0.exe`, `Video-Downloader-Setup-1.2.0.exe.blockmap` and `latest.yml` (these two let installed apps update themselves), plus the `.dmg` if wanted.
5. Click **Publish release** (not "Save draft").

Share link: https://github.com/talal4a/video-downloader/releases/latest

Updating a PC that already has the app

Automatic (best): publish the new release with all three files (`.exe`, `.exe.blockmap`, `latest.yml`). The installed app checks 15 seconds after it starts and every 6 hours (or **Settings → About → Check now**), downloads the update in the background and shows "A new version is ready" → **Restart now**. Nothing else to do on that PC.

By hand: send the new `Video-Downloader-Setup-<version>.exe`; double-clicking it installs over the old version. Settings, history and downloaded files are kept.

- The version number in `package.json` must be higher than the installed one, otherwise nothing updates.
- Version 1.0.0 was published with only the `.exe`, which is fine: the app looks at the newest release, so 1.1.0 just needs all three files.

Automatic build (when GitHub Actions works)

`.github/workflows/build.yml` is ready:

- Every push and pull request: lint, typecheck, unit tests, UI build (Ubuntu).
- `main`, version tags and manual runs: on real Windows it downloads the tools, runs the integration tests, builds the installer, tests the installed app end-to-end, and uploads the installer and screenshots.
- Pushing a tag `v1.2.3` publishes a GitHub Release with the installer, `latest.yml` and blockmap.

Installing on a Windows PC

1. Download `Video-Downloader-Setup-<version>.exe` and double-click it.
2. "Windows protected your PC" → **More info** → **Run anyway** (first time only).
3. It installs to the user's folder and opens. Files are saved in **Downloads → Video Downloader**.
4. Needs normal 64-bit Windows 10 or 11 (not "S mode").

Known limits

Limit	Why / workaround
4K/2K files don't open in QuickTime (Mac)	YouTube only has VP9 at that size. Use VLC or IINA, or pick 1080p
YouTube "not a robot" check	Happens on cloud servers and sometimes busy networks. Wait or restart the router

Limit	Why / workaround
Age-restricted YouTube videos	Need a signed-in YouTube account; Google blocks sign-in inside apps
DRM (Netflix, Prime, Disney+, Spotify)	Not possible and not allowed
Live streams	Download after the stream ends
Some Instagram/Facebook/TikTok/X videos	Log in once in Settings → Accounts
App self-update	Needs a public repository and releases that include `latest.yml`
"Always visible" captions on 4K	Re-encoding 4K takes a long time on older computers; "In the video" is instant

> Only download videos you own or have permission to keep, and respect each site's terms and copyright law.